Nutan Maharashtra Institute of Engineering and Technology (NMIET)
Computer Science and Engineering Department

OOP Practical Programs - C++ Solutions

Source code and sample output for all 22 programs

No.	Topic
1	Student class
2	Student - percentage
3	Employee display
4	Employee array, salary > 5000
5	Rectangle area
6	BankAccount
7	Book display
8	Book array, price > 50
9	Student - highest percentage
10	Employee - total salary (DA, HRA)
11	Method overloading (shape)
12	Mobile class
13	Constructor overloading (shape)
14	Parameterized constructor (Rectangle)
15	Student activity: struct + enum
16	Pointer to derived class
17	Diamond problem, virtual base class
18	Function template swap
19	Server / Client inheritance
20	File handling
21	Multilevel inheritance
22	Single inheritance

Note: compiled with g++ and run to capture the output. Values typed by the user appear after each prompt.

Program 1

Write a C++ program to create a class Student with data members for roll number, name, and marks. Create an object to accept and display student details.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollno;
    string name;
    float marks;
public:
    void accept() {
        cout << "Enter roll number: ";
        cin >> rollno;
        cin.ignore();
        cout << "Enter name: ";
        getline(cin, name);
        cout << "Enter marks: ";
        cin >> marks;
    }
    void display() {
        cout << "\n--- Student Details ---\n";
        cout << "Roll No : " << rollno << endl;
        cout << "Name    : " << name << endl;
        cout << "Marks   : " << marks << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.display();
    return 0;
}
```

Output:

```
Enter roll number: 101
Enter name: Amit Patil
Enter marks: 85.5

--- Student Details ---
Roll No : 101
Name    : Amit Patil
Marks   : 85.5
```

Program 2

Write a program create a class Student having data member rollno, name and marks of two subject. Accept this record for one object of class and calculate the percentage and display all records.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollno;
    string name;
    float m1, m2, percentage;
public:
    void accept() {
        cout << "Enter roll number: ";
        cin >> rollno;
        cin.ignore();
        cout << "Enter name: ";
        getline(cin, name);
        cout << "Enter marks of subject 1 (out of 100): ";
        cin >> m1;
        cout << "Enter marks of subject 2 (out of 100): ";
```

```
        cin >> m2;
    }
    void calculate() {
        percentage = (m1 + m2) / 200 * 100;
    }
    void display() {
        cout << "\n--- Student Record ---\n";
        cout << "Roll No    : " << rollno << endl;
        cout << "Name       : " << name << endl;
        cout << "Subject 1  : " << m1 << endl;
        cout << "Subject 2  : " << m2 << endl;
        cout << "Percentage : " << percentage << "%" << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.calculate();
    s.display();
    return 0;
}
```

Output:

```
Enter roll number: 102
Enter name: Neha Kulkarni
Enter marks of subject 1 (out of 100): 78
Enter marks of subject 2 (out of 100): 88

--- Student Record ---
Roll No    : 102
Name       : Neha Kulkarni
Subject 1  : 78
Subject 2  : 88
Percentage : 83%
```

Program 3

Write a C++ program to create a class Employee with data members for employee ID, name, and salary. Display the employee details using an object.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    int empId;
    string name;
    float salary;
public:
    Employee(int id, string n, float s) {
        empId = id;
        name = n;
        salary = s;
    }
    void display() {
        cout << "--- Employee Details ---\n";
        cout << "Employee ID : " << empId << endl;
        cout << "Name        : " << name << endl;
        cout << "Salary      : " << salary << endl;
    }
};

int main() {
    Employee e(501, "Rahul Sharma", 45000.50);
    e.display();
    return 0;
}
```

Output:

```
--- Employee Details ---
Employee ID : 501
Name        : Rahul Sharma
```

```
Salary      : 45000.5
```

Program 4

Write a program to create a class employee having data members as emp_name, emp_id and salary. Accept this record for 5 objects of class and display that record whose salary is greater than 5000.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    string emp_name;
    int emp_id;
    float salary;
public:
    void accept() {
        cout << "Enter employee name: ";
        cin >> ws;
        getline(cin, emp_name);
        cout << "Enter employee id: ";
        cin >> emp_id;
        cout << "Enter salary: ";
        cin >> salary;
    }
    void display() {
        cout << emp_id << "\t" << emp_name << "\t\t" << salary << endl;
    }
    float getSalary() { return salary; }
};

int main() {
    Employee e[5];
    for (int i = 0; i < 5; i++) {
        cout << "\nEmployee " << i + 1 << ":\n";
        e[i].accept();
    }
    cout << "\nEmployees with salary greater than 5000:\n";
    cout << "ID\tName\t\tSalary\n";
    for (int i = 0; i < 5; i++) {
        if (e[i].getSalary() > 5000)
            e[i].display();
    }
    return 0;
}
```

Output:

```
Employee 1:
Enter employee name: Ravi Kumar
Enter employee id: 1
Enter salary: 4500

Employee 2:
Enter employee name: Sneha Joshi
Enter employee id: 2
Enter salary: 7200

Employee 3:
Enter employee name: Amit Desai
Enter employee id: 3
Enter salary: 5000

Employee 4:
Enter employee name: Pooja Nair
Enter employee id: 4
Enter salary: 12000

Employee 5:
Enter employee name: Suresh Rao
Enter employee id: 5
Enter salary: 3000

Employees with salary greater than 5000:
```

```
ID      Name            Salary
2       Sneha Joshi             7200
4       Pooja Nair              12000
```

Program 5

Write a C++ program to create a class Rectangle with data members for length and breadth. Calculate and display the area using an object.

Code:

```
#include <iostream>
using namespace std;

class Rectangle {
    float length, breadth;
public:
    void accept() {
        cout << "Enter length : ";
        cin >> length;
        cout << "Enter breadth: ";
        cin >> breadth;
    }
    float area() {
        return length * breadth;
    }
    void display() {
        cout << "Area of rectangle = " << area() << endl;
    }
};

int main() {
    Rectangle r;
    r.accept();
    r.display();
    return 0;
}
```

Output:

```
Enter length : 12.5
Enter breadth: 4
Area of rectangle = 50
```

Program 6

Write a C++ program to create a class BankAccount with account number and balance. Accept and display the account details using an object.

Code:

```
#include <iostream>
using namespace std;

class BankAccount {
    long accNo;
    float balance;
public:
    void accept() {
        cout << "Enter account number: ";
        cin >> accNo;
        cout << "Enter balance: ";
        cin >> balance;
    }
    void display() {
        cout << "\n--- Account Details ---\n";
        cout << "Account Number : " << accNo << endl;
        cout << "Balance        : Rs. " << balance << endl;
    }
};

int main() {
    BankAccount b;
    b.accept();
    b.display();
```

```
    return 0;
}
```

Output:

```
Enter account number: 1234567890
Enter balance: 25000.75

--- Account Details ---
Account Number : 1234567890
Balance        : Rs. 25000.8
```

Program 7

Write a C++ program to create a class Book with data members for book title, author, and price. Display the book details using an object.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Book {
    string title, author;
    float price;
public:
    Book(string t, string a, float p) {
        title = t;
        author = a;
        price = p;
    }
    void display() {
        cout << "--- Book Details ---\n";
        cout << "Title  : " << title << endl;
        cout << "Author : " << author << endl;
        cout << "Price  : " << price << endl;
    }
};

int main() {
    Book b("The C++ Programming Language", "Bjarne Stroustrup", 799.0);
    b.display();
    return 0;
}
```

Output:

```
--- Book Details ---
Title  : The C++ Programming Language
Author : Bjarne Stroustrup
Price  : 799
```

Program 8

Write a program to create a class Book having data members as title, Author and price. Accept this record for 5 objects of class and display that record whose price is greater than 50.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Book {
    string title, author;
    float price;
public:
    void accept() {
        cout << "Enter title : ";
        cin >> ws;
        getline(cin, title);
        cout << "Enter author: ";
        getline(cin, author);
        cout << "Enter price : ";
        cin >> price;
```

```
    }
    void display() {
        cout << title << "\t" << author << "\t" << price << endl;
    }
    float getPrice() { return price; }
};

int main() {
    Book b[5];
    for (int i = 0; i < 5; i++) {
        cout << "\nBook " << i + 1 << ":\n";
        b[i].accept();
    }
    cout << "\nBooks with price greater than 50:\n";
    cout << "Title\tAuthor\tPrice\n";
    for (int i = 0; i < 5; i++) {
        if (b[i].getPrice() > 50)
            b[i].display();
    }
    return 0;
}
```

Output:

```
Book 1:
Enter title : Let Us C
Enter author: Yashavant Kanetkar
Enter price : 350

Book 2:
Enter title : Pocket Guide
Enter author: John Doe
Enter price : 40

Book 3:
Enter title : OOP with C++
Enter author: Balaguruswamy
Enter price : 520

Book 4:
Enter title : Short Notes
Enter author: A. Verma
Enter price : 50

Book 5:
Enter title : Data Structures
Enter author: Seymour Lipschutz
Enter price : 275

Books with price greater than 50:
Title   Author  Price
Let Us C        Yashavant Kanetkar      350
OOP with C++    Balaguruswamy   520
Data Structures Seymour Lipschutz       275
```

Program 9

Write a program to create a class student having data members as rollno, name and marks of two subject. Accept this record for 2 objects of class and calculate percentage and display that record whose percentage is highest.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollno;
    string name;
    float m1, m2, percentage;
public:
    void accept() {
        cout << "Enter roll number: ";
        cin >> rollno;
        cout << "Enter name: ";
        cin >> ws;
```

```
        getline(cin, name);
        cout << "Enter marks of subject 1: ";
        cin >> m1;
        cout << "Enter marks of subject 2: ";
        cin >> m2;
        percentage = (m1 + m2) / 200 * 100;
    }
    void display() {
        cout << "Roll No    : " << rollno << endl;
        cout << "Name       : " << name << endl;
        cout << "Subject 1  : " << m1 << endl;
        cout << "Subject 2  : " << m2 << endl;
        cout << "Percentage : " << percentage << "%" << endl;
    }
    float getPercentage() { return percentage; }
};

int main() {
    Student s[2];
    for (int i = 0; i < 2; i++) {
        cout << "\nStudent " << i + 1 << ":\n";
        s[i].accept();
    }
    cout << "\n--- Student with highest percentage ---\n";
    if (s[0].getPercentage() >= s[1].getPercentage())
        s[0].display();
    else
        s[1].display();
    return 0;
}
```

Output:

```
Student 1:
Enter roll number: 1
Enter name: Aarav Mehta
Enter marks of subject 1: 70
Enter marks of subject 2: 80

Student 2:
Enter roll number: 2
Enter name: Isha Gupta
Enter marks of subject 1: 90
Enter marks of subject 2: 85

--- Student with highest percentage ---
Roll No    : 2
Name       : Isha Gupta
Subject 1  : 90
Subject 2  : 85
Percentage : 87.5%
```

Program 10

Write a Program to declare class Employee having data member empid, name, basic salary. Accept this record for two objects of class and calculate Total Salary of Employee where DA is 10% of basic Salary and HRA is 20% basic Salary. (Total Salary = DA + HRA + basic Salary)

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    int empid;
    string name;
    float basic, da, hra, total;
public:
    void accept() {
        cout << "Enter employee id: ";
        cin >> empid;
        cout << "Enter name: ";
        cin >> ws;
        getline(cin, name);
        cout << "Enter basic salary: ";
```

```
        cin >> basic;
    }
    void calculate() {
        da = 0.10 * basic;
        hra = 0.20 * basic;
        total = basic + da + hra;
    }
    void display() {
        cout << "Emp ID       : " << empid << endl;
        cout << "Name         : " << name << endl;
        cout << "Basic Salary : " << basic << endl;
        cout << "DA (10%)     : " << da << endl;
        cout << "HRA (20%)    : " << hra << endl;
        cout << "Total Salary : " << total << endl;
    }
};

int main() {
    Employee e[2];
    for (int i = 0; i < 2; i++) {
        cout << "\nEmployee " << i + 1 << ":\n";
        e[i].accept();
        e[i].calculate();
    }
    cout << "\n===== Salary Details =====\n";
    for (int i = 0; i < 2; i++) {
        cout << "\nEmployee " << i + 1 << ":\n";
        e[i].display();
    }
    return 0;
}
```

Output:

```
Employee 1:
Enter employee id: 201
Enter name: Kiran Pawar
Enter basic salary: 20000

Employee 2:
Enter employee id: 202
Enter name: Meera Shah
Enter basic salary: 35000

===== Salary Details =====

Employee 1:
Emp ID       : 201
Name         : Kiran Pawar
Basic Salary : 20000
DA (10%)     : 2000
HRA (20%)    : 4000
Total Salary : 26000

Employee 2:
Emp ID       : 202
Name         : Meera Shah
Basic Salary : 35000
DA (10%)     : 3500
HRA (20%)    : 7000
Total Salary : 45500
```

Program 11

Write a Program to declare class shape then calculate Area of Circle, Area of Triangle, Area of Rectangle, Area of Square using Method overloading.

Code:

```
#include <iostream>
using namespace std;

class shape {
public:
    // Circle: one float argument (radius)
    float area(float r) {
```

```
        return 3.14159 * r * r;
    }
    // Square: one int argument (side)
    int area(int side) {
        return side * side;
    }
    // Rectangle: two float arguments (length, breadth)
    float area(float l, float b) {
        return l * b;
    }
    // Triangle: two int arguments (base, height)
    float area(int b, int h) {
        return 0.5 * b * h;
    }
};

int main() {
    shape s;
    cout << "Area of Circle (r = 5.0)         : " << s.area(5.0f) << endl;
    cout << "Area of Square (side = 4)        : " << s.area(4) << endl;
    cout << "Area of Rectangle (6.5 x 3.2)    : " << s.area(6.5f, 3.2f) << endl;
    cout << "Area of Triangle (b = 8, h = 5)  : " << s.area(8, 5) << endl;
    return 0;
}
```

Output:

```
Area of Circle (r = 5.0)         : 78.5397
Area of Square (side = 4)        : 16
Area of Rectangle (6.5 x 3.2)    : 20.8
Area of Triangle (b = 8, h = 5)  : 20
```

Program 12

Write a Program to declare class Mobile with data member 'Company_name' & 'Screen Size'. Accept this record for one object of class and display it.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Mobile {
    string Company_name;
    float Screen_Size;
public:
    void accept() {
        cout << "Enter company name: ";
        getline(cin, Company_name);
        cout << "Enter screen size (inches): ";
        cin >> Screen_Size;
    }
    void display() {
        cout << "\n--- Mobile Details ---\n";
        cout << "Company Name : " << Company_name << endl;
        cout << "Screen Size  : " << Screen_Size << " inches" << endl;
    }
};

int main() {
    Mobile m;
    m.accept();
    m.display();
    return 0;
}
```

Output:

```
Enter company name: Samsung
Enter screen size (inches): 6.5

--- Mobile Details ---
Company Name : Samsung
Screen Size  : 6.5 inches
```

Program 13

Write a program to create class shape and calculate area of circle, Rectangle, square and triangle using concept of constructor overloading.

Code:

```
#include <iostream>
using namespace std;

class shape {
    float area;
    const char *type;
public:
    // Circle: (double radius)
    shape(double r) {
        area = 3.14159 * r * r;
        type = "Circle";
    }
    // Square: (int side)
    shape(int side) {
        area = side * side;
        type = "Square";
    }
    // Rectangle: (double length, double breadth)
    shape(double l, double b) {
        area = l * b;
        type = "Rectangle";
    }
    // Triangle: (int base, int height)
    shape(int b, int h) {
        area = 0.5 * b * h;
        type = "Triangle";
    }
    void display() {
        cout << "Area of " << type << " = " << area << endl;
    }
};

int main() {
    shape c(5.0);        // circle, radius 5.0
    shape s(4);          // square, side 4
    shape r(6.5, 3.2);   // rectangle, 6.5 x 3.2
    shape t(8, 5);       // triangle, base 8, height 5
    c.display();
    s.display();
    r.display();
    t.display();
    return 0;
}
```

Output:

```
Area of Circle = 78.5397
Area of Square = 16
Area of Rectangle = 20.8
Area of Triangle = 20
```

Program 14

Write a program to create a class Rectangle having data member's length and width initialized it using parameterized constructors for 2 objects of class and calculate area of rectangle and display it.

Code:

```
#include <iostream>
using namespace std;

class Rectangle {
    float length, width;
public:
    Rectangle(float l, float w) {   // parameterized constructor
        length = l;
        width = w;
    }
    float area() {
```

```
        return length * width;
    }
    void display() {
        cout << "Length = " << length << ", Width = " << width
             << ", Area = " << area() << endl;
    }
};

int main() {
    Rectangle r1(10, 5);
    Rectangle r2(7.5, 4.2);
    cout << "Rectangle 1: ";
    r1.display();
    cout << "Rectangle 2: ";
    r2.display();
    return 0;
}
```

Output:

```
Rectangle 1: Length = 10, Width = 5, Area = 50
Rectangle 2: Length = 7.5, Width = 4.2, Area = 31.5
```

Program 15

Develop a C++ program to track student activity participation using basic OOP concepts. Represent student details (name, roll number, division, activity score) using appropriate data types, and store activity information using a struct Activity. Use an enum ParticipationStatus to indicate whether a student has NotParticipated, Participated, or Completed an activity. Accept all inputs using suitable control structures and display a formatted summary of student details, activity details, and participation status. Implement the solution using a Student class with relevant data members and member functions.

Code:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

enum ParticipationStatus { NotParticipated, Participated, Completed };

struct Activity {
    string activityName;
    int durationHours;
    ParticipationStatus status;
};

class Student {
    string name;
    int rollNo;
    char division;
    float activityScore;
    Activity activity;
public:
    void accept() {
        int choice;
        cout << "Enter student name: ";
        cin >> ws;
        getline(cin, name);
        cout << "Enter roll number: ";
        cin >> rollNo;
        cout << "Enter division (A/B/C): ";
        cin >> division;
        cout << "Enter activity score: ";
        cin >> activityScore;
        cout << "Enter activity name: ";
        cin >> ws;
        getline(cin, activity.activityName);
        cout << "Enter duration (hours): ";
        cin >> activity.durationHours;
        do {
            cout << "Status (0-NotParticipated, 1-Participated, 2-Completed): ";
            cin >> choice;
        } while (choice < 0 || choice > 2);
        activity.status = static_cast<ParticipationStatus>(choice);
```

```
    }
    string statusText() {
        switch (activity.status) {
            case NotParticipated: return "Not Participated";
            case Participated:    return "Participated";
            case Completed:       return "Completed";
        }
        return "Unknown";
    }
    void display() {
        cout << left << setw(12) << name
             << setw(6)  << rollNo
             << setw(6)  << division
             << setw(8)  << activityScore
             << setw(16) << activity.activityName
             << setw(8)  << activity.durationHours
             << statusText() << endl;
    }
};

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    Student s[10];
    for (int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << ":\n";
        s[i].accept();
    }
    cout << "\n=========== Activity Participation Summary ===========\n";
    cout << left << setw(12) << "Name" << setw(6) << "Roll"
         << setw(6) << "Div" << setw(8) << "Score"
         << setw(16) << "Activity" << setw(8) << "Hours"
         << "Status" << endl;
    cout << string(70, '-') << endl;
    for (int i = 0; i < n; i++)
        s[i].display();
    return 0;
}
```

Output:

```
Enter number of students: 3

Student 1:
Enter student name: Anita
Enter roll number: 1
Enter division (A/B/C): A
Enter activity score: 8.5
Enter activity name: Quiz
Enter duration (hours): 2
Status (0-NotParticipated, 1-Participated, 2-Completed): 2

Student 2:
Enter student name: Bhushan
Enter roll number: 2
Enter division (A/B/C): B
Enter activity score: 6
Enter activity name: Debate
Enter duration (hours): 3
Status (0-NotParticipated, 1-Participated, 2-Completed): 1

Student 3:
Enter student name: Chaitali
Enter roll number: 3
Enter division (A/B/C): A
Enter activity score: 0
Enter activity name: Robotics
Enter duration (hours): 10
Status (0-NotParticipated, 1-Participated, 2-Completed): 0

=========== Activity Participation Summary ===========
Name        Roll  Div   Score   Activity        Hours   Status
----------------------------------------------------------------------
Anita       1     A     8.5     Quiz            2       Completed
Bhushan     2     B     6       Debate          3       Participated
Chaitali    3     A     0       Robotics        10      Not Participated
```

Program 16

Pointer to derived class in single inheritance. Pointer to derived class in multilevel inheritance.

Code:

```
#include <iostream>
using namespace std;

// ---------- Single inheritance ----------
class Base {
public:
    int b;
    void showBase() { cout << "Base class: b = " << b << endl; }
};

class Derived : public Base {
public:
    int d;
    void showDerived() { cout << "Derived class: d = " << d << endl; }
};

// ---------- Multilevel inheritance ----------
class A {
public:
    int a;
    void showA() { cout << "Class A: a = " << a << endl; }
};

class B : public A {
public:
    int b;
    void showB() { cout << "Class B: b = " << b << endl; }
};

class C : public B {
public:
    int c;
    void showC() { cout << "Class C: c = " << c << endl; }
};

int main() {
    cout << "===== Single Inheritance =====\n";
    Derived obj;
    Derived *dptr = &obj;          // pointer to derived class
    dptr->b = 10;                  // base member through derived pointer
    dptr->d = 20;
    dptr->showBase();
    dptr->showDerived();

    Base *bptr = &obj;             // base pointer to derived object
    cout << "Using base pointer: ";
    bptr->showBase();

    cout << "\n===== Multilevel Inheritance =====\n";
    C objC;
    C *cptr = &objC;               // pointer to derived (class C)
    cptr->a = 1;
    cptr->b = 2;
    cptr->c = 3;
    cptr->showA();
    cptr->showB();
    cptr->showC();

    B *bp = &objC;                 // pointer to intermediate class
    A *ap = &objC;                 // pointer to root class
    cout << "Using pointer of class B: ";
    bp->showB();
    cout << "Using pointer of class A: ";
    ap->showA();
    return 0;
}
```

Output:

```
===== Single Inheritance =====
Base class: b = 10
Derived class: d = 20
Using base pointer: Base class: b = 10

===== Multilevel Inheritance =====
Class A: a = 1
Class B: b = 2
Class C: c = 3
Using pointer of class B: Class B: b = 2
Using pointer of class A: Class A: a = 1
```

Program 17

Write a C++ program to demonstrate the Diamond Problem in multiple inheritance and implement its resolution using virtual base classes.

Code:

```
#include <iostream>
using namespace std;

class Person {                        // common base (top of diamond)
public:
    string name;
    Person() { cout << "Person constructor called\n"; }
};

// 'virtual' ensures only ONE copy of Person is inherited
class Teacher : virtual public Person {
public:
    Teacher() { cout << "Teacher constructor called\n"; }
};

class Researcher : virtual public Person {
public:
    Researcher() { cout << "Researcher constructor called\n"; }
};

class Professor : public Teacher, public Researcher {
public:
    Professor() { cout << "Professor constructor called\n"; }
    void show() {
        // Without virtual: "request for member 'name' is ambiguous" error
        cout << "Professor name: " << name << endl;
    }
};

int main() {
    Professor p;
    p.name = "Dr. Shital Mehta";
    p.show();
    cout << "Size of Professor object: " << sizeof(p) << " bytes\n";
    return 0;
}
```

Output:

```
Person constructor called
Teacher constructor called
Researcher constructor called
Professor constructor called
Professor name: Dr. Shital Mehta
Size of Professor object: 48 bytes
```

Program 18

Function template to swap integers and floats. Write a C++ program to swap two integer values and two float values by using function template.

Code:

```
#include <iostream>
using namespace std;
template <class T>
```

```
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int i1, i2;
    float f1, f2;

    cout << "Enter two integers: ";
    cin >> i1 >> i2;
    cout << "Enter two floats  : ";
    cin >> f1 >> f2;

    cout << "\nBefore swap: i1 = " << i1 << ", i2 = " << i2 << endl;
    swapValues(i1, i2);
    cout << "After swap : i1 = " << i1 << ", i2 = " << i2 << endl;

    cout << "\nBefore swap: f1 = " << f1 << ", f2 = " << f2 << endl;
    swapValues(f1, f2);
    cout << "After swap : f1 = " << f1 << ", f2 = " << f2 << endl;
    return 0;
}
```

Output:

```
Enter two integers: 10 20
Enter two floats  : 3.5 7.25

Before swap: i1 = 10, i2 = 20
After swap : i1 = 20, i2 = 10

Before swap: f1 = 3.5, f2 = 7.25
After swap : f1 = 7.25, f2 = 3.5
```

Program 19

Develop a program to create a class "Server" having data member 'serv_name' and 'serv_port' and method show(). Derive a class "Client" having data member 'client_name' and 'Client_port' with method show(). Accept and display records.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Server {
protected:
    string serv_name;
    int serv_port;
public:
    void accept() {
        cout << "Enter server name: ";
        cin >> ws;
        getline(cin, serv_name);
        cout << "Enter server port: ";
        cin >> serv_port;
    }
    void show() {
        cout << "Server Name : " << serv_name << endl;
        cout << "Server Port : " << serv_port << endl;
    }
};

class Client : public Server {
    string client_name;
    int Client_port;
public:
    void accept() {
        Server::accept();
        cout << "Enter client name: ";
        cin >> ws;
        getline(cin, client_name);
        cout << "Enter client port: ";
```

```
        cin >> Client_port;
    }
    void show() {
        Server::show();
        cout << "Client Name : " << client_name << endl;
        cout << "Client Port : " << Client_port << endl;
    }
};

int main() {
    Client c;
    c.accept();
    cout << "\n--- Server and Client Records ---\n";
    c.show();
    return 0;
}
```

Output:

```
Enter server name: WebServer01
Enter server port: 8080
Enter client name: Client-PC-7
Enter client port: 5050

--- Server and Client Records ---
Server Name : WebServer01
Server Port : 8080
Client Name : Client-PC-7
Client Port : 5050
```

Program 20

File creation, writing, closing, reading. Write a C++ program that creates an output file, writes information to it, closes the file and open it again as an input file and read the information from the file.

Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // ----- Step 1: create file and write -----
    ofstream fout("student.txt");
    if (!fout) {
        cout << "Error: could not create file.\n";
        return 1;
    }
    string name, branch;
    int roll;
    cout << "Enter name  : ";
    getline(cin, name);
    cout << "Enter roll  : ";
    cin >> roll;
    cout << "Enter branch: ";
    cin >> ws;
    getline(cin, branch);

    fout << "Name: " << name << endl;
    fout << "Roll: " << roll << endl;
    fout << "Branch: " << branch << endl;
    fout.close();                       // ----- Step 2: close -----
    cout << "\nData written to student.txt and file closed.\n";

    // ----- Step 3: open as input and read -----
    ifstream fin("student.txt");
    if (!fin) {
        cout << "Error: could not open file.\n";
        return 1;
    }
    string line;
    cout << "\nReading from file:\n";
    while (getline(fin, line))
        cout << line << endl;
    fin.close();
```

```
    return 0;
}
```

Output:

```
Enter name  : Prasad Jadhav
Enter roll  : 45
Enter branch: Computer Science

Data written to student.txt and file closed.

Reading from file:
Name: Prasad Jadhav
Roll: 45
Branch: Computer Science
```

(This program also creates the file student.txt in the working folder.)

Program 21

Write a simple C++ program to demonstrate multilevel inheritance using three classes: Student, Marks, and Result. Class Student stores the student's roll number and name. Class Marks inherits from Student and stores marks. Class Result inherits from Marks and displays the student's details, marks, and total.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
protected:
    int rollno;
    string name;
public:
    void getStudent() {
        cout << "Enter roll number: ";
        cin >> rollno;
        cout << "Enter name: ";
        cin >> ws;
        getline(cin, name);
    }
};

class Marks : public Student {
protected:
    float m1, m2, m3;
public:
    void getMarks() {
        cout << "Enter marks of 3 subjects: ";
        cin >> m1 >> m2 >> m3;
    }
};

class Result : public Marks {
    float total;
public:
    void display() {
        total = m1 + m2 + m3;
        cout << "\n--- Result ---\n";
        cout << "Roll No : " << rollno << endl;
        cout << "Name    : " << name << endl;
        cout << "Marks   : " << m1 << ", " << m2 << ", " << m3 << endl;
        cout << "Total   : " << total << endl;
    }
};

int main() {
    Result r;
    r.getStudent();
    r.getMarks();
    r.display();
    return 0;
}
```

Output:

```
Enter roll number: 31
Enter name: Simran Khan
Enter marks of 3 subjects: 78 82 91

--- Result ---
Roll No : 31
Name    : Simran Khan
Marks   : 78, 82, 91
Total   : 251
```

Program 22

Write a simple C++ program to demonstrate single inheritance using two classes: Person and Student. Class Person stores the person's name and age. Class Student inherits from Person and stores the student's roll number. Display all details using an object of the Student class.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int age;
public:
    void getPerson() {
        cout << "Enter name: ";
        getline(cin, name);
        cout << "Enter age: ";
        cin >> age;
    }
};

class Student : public Person {
    int rollno;
public:
    void getStudent() {
        getPerson();
        cout << "Enter roll number: ";
        cin >> rollno;
    }
    void display() {
        cout << "\n--- Student Details ---\n";
        cout << "Name    : " << name << endl;
        cout << "Age     : " << age << endl;
        cout << "Roll No : " << rollno << endl;
    }
};

int main() {
    Student s;
    s.getStudent();
    s.display();
    return 0;
}
```

Output:

```
Enter name: Rohan Deshmukh
Enter age: 19
Enter roll number: 23

--- Student Details ---
Name    : Rohan Deshmukh
Age     : 19
Roll No : 23
```